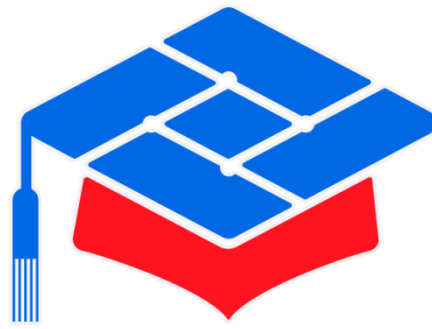




第十届

全国大学生集成电路创新创业大赛

报告类型： 技术文档

参赛杯赛： “紫光同创” 企业命题

作品名称： 基于 FPGA 和脉动卷积阵列的 YOLO、LPRNet 级联网络实
现的实时车牌检测系统

队伍编号： CICC1000270

团队名称： 电子榨菜队

目录

- 1. 快速预览简介.....1
 - 1.1 核心关键技术.....1
 - 1.2 作品亮点.....1
 - 1.3 系统关键指标.....2
- 2. 系统总体设计方案.....3
- 3. 算法流程图.....4
- 4. 核心代码.....6
- 5. 仿真与测试报告.....8
 - 5.1 模型结构.....8
 - 5.1.1 YOLO 车牌检测模型.....8
 - 5.1.2 LPRNet 车牌号识别模型.....8
 - 5.2 浮点模型性能测试.....9
 - 5.2.1 YOLO 车牌检测精度.....9
 - 5.2.2 LPRNet 车牌号识别精度.....10
 - 5.3 量化误差分析.....11
 - 5.3.1 量化框架与配置.....11
 - 5.3.2 YOLO 车牌检测模型量化误差分析.....11
 - 5.3.3 LPRNet 车牌号识别模型量化误差分析.....12
 - 5.4 Modelsim 仿真.....14
 - 5.4.1 卷积计算模块.....14
 - 5.4.2 子图裁切缩放模块.....15
 - 5.5 实际测试.....16
 - 5.5.1 延迟与帧率.....16
 - 5.5.2 光强适应性.....16
 - 5.5.3 大小、角度适应性.....17
 - 5.5.4 同步检测数量.....18
 - 5.5.5 正确率.....18
 - 5.5.6 资源消耗.....18

5.7 时序报告	20
5.8 功耗报告	21
6. AI 工具使用说明	22

基于 FPGA 和脉动卷积阵列 YOLO、LPRNet 级联网络实现的实时车牌检测系统

1. 快速预览简介

本作品针对边缘端机器视觉应用中对低延迟、高吞吐和低功耗的严苛要求，设计并实现了一个基于 FPGA（PG2L200H）的端到端实时车牌检测与识别系统。系统完全摒弃了传统的软硬协同或外挂缓存方案，通过全流水线的纯硬件逻辑实现从视频流输入、双神经网络级联推理到结果渲染输出的“即入即算即显”全闭环。

1.1 核心关键技术

异构双模型级联硬件推理：系统在片上级联部署了深度轻量化后的目标定位模型（YOLO）与序列识别模型（LPRNet）。在 RTL 底层，两个异构网络复用了高度参数化的通用层级加速模块，实现了计算资源的高效复用与动态调配。

脉动阵列全流水线卷积架构：卷积核心创新性地采用了三维结构、分时复用的脉动阵列设计，基础处理单元阵列结构规整，数据复用率极高，在解决“内存墙”带宽瓶颈的同时，优化了布局布线时序，显著提升了系统的最高工作频率与推理吞吐量。网络层与层之间通过分布式的 Line Buffer 实现线连直通，特征数据以“全流水线”形态在阵列中流动，无需任何软件调度开销。

1.2 作品亮点

算法与底层 RTL 的深度协同优化：软件上的多种类型 bit 量化、模型分支裁剪，配合硬件上的优秀流水线卷积架构设计，带来了高准确率、高鲁棒性，同时具有低延迟、低功耗的特点。

高鲁棒性的复杂物理场景适应能力：系统不仅在理论与仿真中表现优异，在实际上板测试中也展现出强大的工程鲁棒性。能够稳定适应强光、弱光、逆光等复杂光照环境，并对 20x80 至 200x800 的多尺度车牌及高达 30° 的多维倾斜角（俯仰、偏航、横滚）具有良好的抗干扰与识别能力。

1.3 系统关键指标

指标维度	实测数据与性能表现
处理速度与延迟	在 148.5MHz 工作频率下，YOLO 仿真延迟低至 1.14ms，LPRNet 延迟仅 2.35ms 。系统可在单帧（720P@60Hz）内完成 多个车牌的同步定位与识别，单帧内完成 8 个车牌号的识别，端到端延迟达毫秒级。
识别准确率	量化信噪比误差极低（YOLO < 0.05%，LPRNet < 0.06%）。在真实场景高清晰度下，车牌类型检测正确率近乎 100%，车牌号识别正确率达 95% 以上 。
硬件资源消耗	资源利用率极佳且留有余量：LUT 消耗 10.65%，Registers 消耗 27.26%，DRM 消耗 47.11%，APM 消耗 53.51%。720P@60Hz 所要求的时序完全收敛。
能效与功耗	系统总功耗仅为 3.377W （其中静态功耗 0.825W），相比传统 CPU/GPU 方案具有压倒性的能效优势。

表 1.1 系统关键指标

2. 系统总体设计方案

本系统的所有计算均在 PG2L200H 芯片上完成，将 HDMI 信号输入中的车牌类型、位置、大小和车牌号序列信息用片上的两个卷积神经网络进行提取，处理结果再叠加到视频流上并输出到 HDMI 输出端口。

本系统采用基于**流式数据流驱动(Streaming Dataflow)**的端到端实时视频流处理架构。系统整体划分为四大核心子系统：**视频接入与同步模块**、**异构双网络推理加速模块**、**动态 ROI 裁剪与图像处理模块**，以及 **OSD 融合与视频输出模块**。系统各核心模块的设计原理及特点如下：

异构双模型级联推理机制 系统在片上级联部署了用于目标定位的车牌检测模型（YOLO）与用于序列识别的字符解码模型（LPRNet）。算法侧经过深度轻量化（定点量化、算子融合与通道裁剪），仅保留核心的卷积、池化及硬件友好的后处理逻辑。在 RTL 底层，两个异构网络复用了高度参数化、高聚合度的通用层级加速 IP，通过脚本化配置实现了资源的动态复用与灵活调配。

全流水线脉动阵列卷积架构 有别于依赖片外 DDR 搬运中间特征图的传统指令驱动型脉动阵列，本设计的卷积核心采用了三维空间与时间复用展开的方式。网络层与层之间通过分布式的 Line Buffer 与 Ping-Pong RAM 实现线连直通，特征数据以“全流水线”形态在阵列中流动。该架构基础 PE（处理单元）阵列结构规整，数据复用率极高，在彻底消除“内存墙”带宽瓶颈的同时，优化了布局布线时序，显著提升了系统的最高工作频率与推理吞吐量。

动态特征截取与片上视频渲染 YOLO 网络输出检测坐标后，系统通过边界框同步模块（Box Overlay Sync）在原始视频流中实时绘制目标框，并联动子图缓存管理器（Crop Buffer Manager）对感兴趣区域（ROI）进行动态裁剪。裁剪出的高清车牌特征经缩放后送入后级 LPRNet 模块进行识别。

“即入即算即显”的低延迟输出 LPRNet 输出字符预测结果后，通过字符叠加模块（Char Overlay）以像素级精度无缝融合至原始视频帧的对应物理坐标系中。融合后的最终视频流通过 RGB 转 TMDS 编码模块输出至 HDMI 接口，全程无需任何外部软件或片外存储介入，实现了极低延迟的端到端机器视觉应用闭环。

3. 算法流程图

系统整体流程

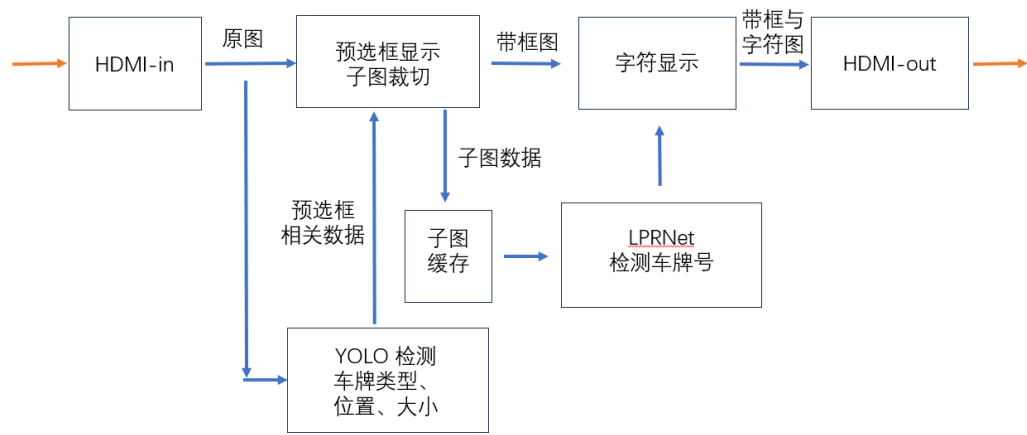


图 3.1 算法整体流程

将 HDMI 输入的原图传输给 YOLO 检测模块，检测原图中的车牌类型、大小和位置，得到预选框相关数据。将预选框数据传给子图裁切模块，裁切子图后经子图缓存模块缩放后由 LPRNet 检测车牌号。检测到的预选框数据和车牌号在原图视频流中用叠加的方式显示框和车牌字符串。两个卷积网络的数据从输入到输出采用全流水线的结构，中间没有任何软件调度的开销。

单层神经网络结构

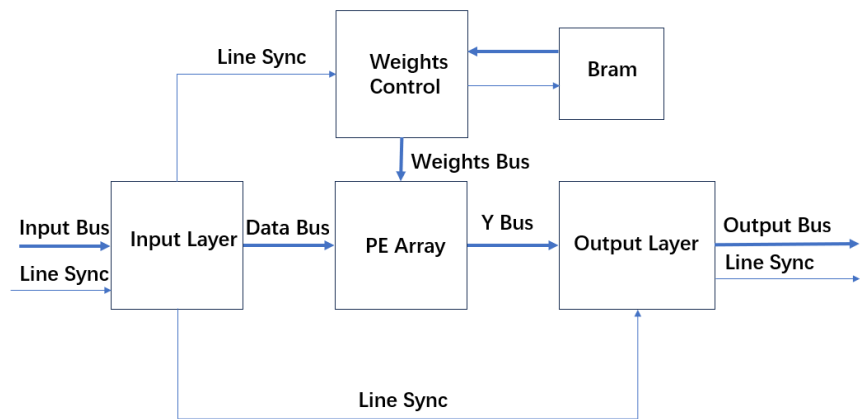


图 3.2 单层神经网络结构

单层的神经网络包括输入层、权重控制层、计算阵列和输出层四个主要部分，他们之间的同步是用行同步的逻辑实现的。其中输入层将串行的输入数据转化为并行的卷积窗口数据，并得到单层内部的行同步模块。权重管理模块将预偏移后的权重转化为卷积窗口中的权重，所有权重存储在 **Bram** 中。卷积计算单元阵列将中间结果传递给输出层，输出层通过累加得到最终输出，并输出出行同步信息给下一层。

三维脉动卷积阵列结构

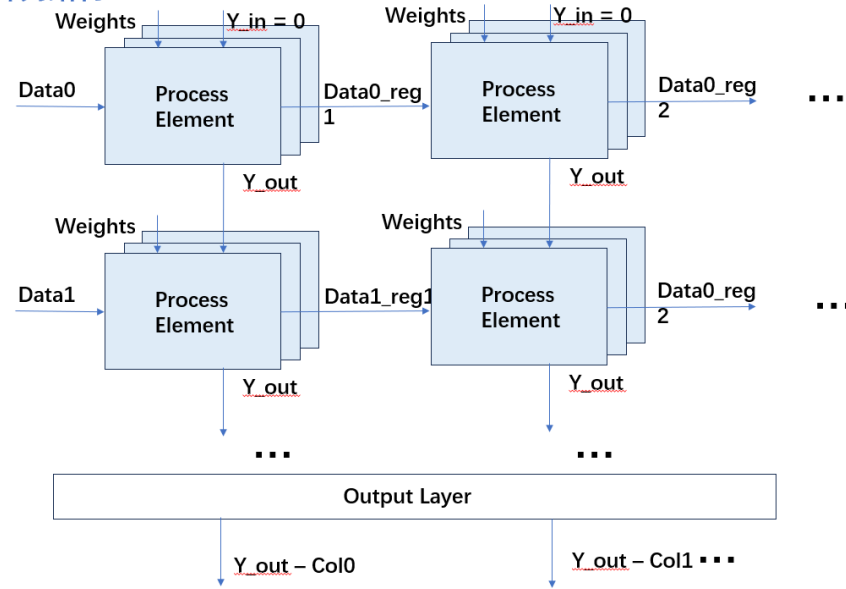


图 3.3 三维脉动卷积阵列计算单元结构

卷积计算单元阵列（PE Array）是用类似脉动阵列卷积的结构实现的。卷积阵列的结构是三维的，每一页是不同的输入通道，每一列是不同的输出通道，每一行是卷积窗口的不同位置。输入数据从左到右流动，权重从每一列的 **Bram** 中读取，计算的中间结果从上到下流动。电路拓扑结构重复度高，单个计算单元仅与相邻模块连接，极大地降低了布线拥塞。

4. 核心代码

核心代码（`data_process` 内的 `layer.sv` 是卷积的单层顶层模块，其他模块也是与卷积相关的核心代码）。单层卷积模块的构成：

1. 数据重组与窗口生成 (Data Formatting)

这一步由 `input_layer.sv` 完成，负责将一维的串行像素流转化为适配卷积核的二维数据块。

行缓冲 (Line Buffering): 输入的像素数据流首先被写入基于 `SDP_RAM` 的行缓存 (Line Buffer) 和乒乓缓存 (Ping-Pong RAM) 中，以此来缓存计算所需的多个完整图像行。

滑动窗口提取: 随后，利用移位寄存器 (Shift Registers) 和列缓存机制，将行缓存中的数据截取为 `KERNEL_ROW x KERNEL_COL` 大小的二维特征窗口。

边界与步长调度: 模块内部维护了图像坐标的索引 (`index_x, index_y`)。当窗口滑出图像边界时，自动执行 `Padding` (补零操作)；同时根据设定的卷积步长 (`STEP_ROW, STEP_COL`)，精准控制有效窗口数据的输出时机。

2. 核心乘加流水计算 (MAC Pipeline)

这一步由 `pe_page.sv` 及其子模块 (`d_manager, w_manager, pe_col, pe`) 完成，负责执行实质的卷积运算。

数据延时对齐: `d_manager` 接收到二维窗口后，为每一行数据施加逐级递增的时钟延迟，形成阶梯状的数据流，以确保数据在正确的时间到达阵列深处的处理单元。

权重同步分发: `w_manager` 依据当前的时钟周期，从本地只读存储器 (`.mem`) 中精准取出对应输出通道和输入通道的权重矩阵。

脉动乘加计算: 数据和权重进入深度级联的 `pe` 阵列。每个 `pe` 单元在一个时钟周期内完成一次乘法与加法，并将产生的部分和 (Partial Sum) 传递给下一个单元。整个过程像水流一样连续不断，无需等待整个卷积核算完。

3. 跨通道聚合与后处理 (Accumulation & Post-Processing)

这一步由 `output_layer.sv` 完成，负责将离散的部分和汇总为最终的特征图通道。

空间与时间累加: 首先，将并行处理的多个输入通道 (`PE_PAGE`) 的部分和在空间上相加。接着，利用移位寄存器构成的反馈环路，在指定周期 (`CYCLE_PERIOD_OUT`) 内完成时间维度的深度累加。

偏置与量化: 将累加完成的完整卷积结果加上预设的偏置值 (Bias)，随后进行算术右移 (`SHIFT_KEY`) 并配合四舍五入，将高位宽数据截断回目标位宽。

激活与对齐输出： 结果通过 ReLU 激活函数（清零负值并处理正向饱和极值）。最后，使用输出延迟链消除各列之间因流水线级数差异造成的时序倾斜，输出完全对齐的有效特征图流。

5. 仿真与测试报告

5.1 模型结构

5.1.1 YOLO 车牌检测模型

本部分采用 Ultralytics 库进行 YOLO 车牌检测模型的训练，所训练的模型结构为简化版的端到端的单头检测模型。考虑到 FPGA 端部署难度，我们将骨干网络简化为五个 Conv 卷积块堆叠，其中 Conv 块为 conv、bn、relu 的级联结构，去除了原有模型中复杂的瓶颈、残差、特征融合等结构；并使用自定义的 FewClsDetect 检测头，由于需要检测的车牌类别仅包含蓝牌、绿牌、单层黄牌、黑牌和白牌五种类别，我们自定义一个 FewClsDetect 检测头，用于强制指定 cls 和 bbox 分支的隐藏层卷积核数量；并且将默认的 SILU 激活函数替换为 RELU 激活函数。采用 YOLO26 最新的端到端训练模式进行训练。

```
# YOLO26n backbone
backbone:
  # [from, repeats, module, args]
  - [-1, 1, Conv, [8, 3, 2]] # 0-P1/2
  - [-1, 1, Conv, [16, 3, 2]] # 1-P2/4
  - [-1, 1, Conv, [32, 3, 2]] # 2-P3/8
  - [-1, 1, Conv, [64, 3, 2]] # 3-P4/16
  - [-1, 1, Conv, [64, 3, 1]] # 4-P4/16
# YOLO26n head
head:
  - [[-1], 1, FewClsDetect, [nc, [32, 32]]] # FewClsDetect(P4)
```

图 5.1 YOLO 卷积神经网络结构

5.1.2 LPRNet 车牌号识别模型

本部分采用开源第三方库 STNet_LPRNet 进行 LPRNet 车牌号识别模型的训练，所训练的模型结构为简化版的 LRPnet 模型。同 YOLO 车牌检测模型，为了降低 FPGA 部署难度，我们去除了原模型中的 small_basic_block 卷积块、多尺度特征融合以及通过精心设计 padding 实现的特征图尺寸缩减策略，仅仅使用 Conv 卷积块，Maxpool 池化层、nn.Conv2d 标准卷积层以及 torch.mean 函数，通过设计输入图像大小及 stride 步长实现特征图尺寸缩减。

```

self.backbone = nn.Sequential( # [bs,3,20,80]
    Conv(c1:3, c2:16, k:3, s:1, p:1), # [bs,16,20,80]
    nn.MaxPool2d(kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)), # [bs,16,20,80]
    Conv(c1:16, c2:32, k:3, s:(1, 2), p:1), # [bs,32,20,40]
    nn.MaxPool2d(kernel_size=(3, 3), stride=(2, 1), padding=(1, 1)), # [bs,32,10,40]
    Conv(c1:32, c2:64, k:3, s:(1, 2), p:1), # [bs,64,10,20]
    nn.MaxPool2d(kernel_size=(3, 3), stride=(2, 1), padding=(1, 1)), # [bs,128,5,20]
    Conv(c1:64, c2:128, k:3, s:1, p:1), # [bs,128,5,20]
    Conv(c1:128, c2:128, k:(1, 3), s:1, p:(0, 1)), # [bs,128,5,20]
)
self.container = nn.Sequential(
    nn.Conv2d(in_channels=128, out_channels=class_num, kernel_size=(1, 1), stride=(1, 1)),
    # [bs,plate_char_cls,4,20]
    # torch.mean(dim=2) # [bs,plate_char_cls,20]
)

```

图 5.2 LPRNet 卷积神经网络结构

5.2 浮点模型性能测试

5.2.1 YOLO 车牌检测精度

YOLO 车牌检测模型训练以及测试数据集采用 CCPD2019 及 CCPD2020 各随机抽取 3750 个样本作为蓝牌、绿牌数据集；由于没有开源的单层黄牌、黑牌以及白牌数据集，我们采用开源第三方库 `chinese_license_plate_generator` 配合 CCPD2019 数据集，通过生成标准车牌、随机数据增加、透射变换再贴回 CCPD 原图的方式分别合成了 2500、1250、1250 个单层黄牌、黑牌以及白牌数据集；按照 8: 1: 1 划分为训练、验证及测试集。

YOLO 训练数据示例如下。



图 5.3 YOLO 数据集示例

YOLO 车牌检测模型精度如下图所示。

Class	Images	Instances	Box(P	R	mAP50	mAP50-95)
all	1250	1250	0.962	0.968	0.99	0.695
blue_plate	375	375	0.936	0.949	0.982	0.669
green_plate	375	375	0.981	0.983	0.994	0.836
yellow_plate	250	250	0.968	0.984	0.994	0.691
black_plate	125	125	0.94	0.96	0.99	0.648
white_plate	125	125	0.984	0.964	0.989	0.63

图 5.4 YOLO 车牌检测精度

5.2.2 LPRNet 车牌号识别精度

LPRNet 车牌号识别模型训练及测试数据集采用 CCPD2019 及 CCPD2020 各抽取 100k、10k 个样本作为蓝牌及绿牌真实数据集；另外采用第三方库 chinese_license_plate_generator 配合 CCPD2019 数据集，通过生成标准车牌、随机数据增强、透射变换、贴回 CCPD 原图再裁剪出矩形区域的方式分别合成了 90k、100k、100k、100k 个绿牌、单层黄牌、黑牌以及白牌合成数据集；按照 8：1：1 划分为训练、验证及测试集，训练及测试过程均采用最邻近插值方法进行图像缩放，以适配 FPGA 图像缩放模块。



图 5.5 LPRNet 训练数据示例

Class	Image	Accuracy
all	50000	0.95872
真实蓝牌	10000	0.9431
真实绿牌	1000	0.885
合成绿牌	9000	0.93756
合成黄牌	10000	0.9771
合成黑牌	10000	0.9678
合成白牌	10000	0.9733

图 5.6 LPRNet 车牌号识别模型精度

5.3 量化误差分析

5.3.1 量化框架与配置

本部分采用开源量化框架 PPQ 对 YOLO 及 LPRNet 模型进行量化。为了最小化 FPGA 部署时层间反量化操作的难度及资源消耗，我们采用了对称及 Power Of Two 的量化策略，该策略使得 FPGA 可以使用移位实现反量化操作，大大降低实现难度以及资源消耗。

5.3.2 YOLO 车牌检测模型量化误差分析

考虑到 FPGA 图像输入格式为 RGB888 并且使得量化模型精度损失可控，我们对输入数据的量化使用 9 比特对称量化，从而实现 FPGA 图像输入无需任何操作直接可用；对其张量量化采用 9 比特对量化，以保证量化误差足够小。

YOLO 车牌检测模型累计信噪比误差分析如下，其中 node_conv2d_7 为 bbox 分支输出、node_conv2d_10 为 cls 分支输出，两个输出的累计误差均小于 0.05%。

layer_name	: SNR(N/S)
node_conv2d	: 0.0010%
node_conv2d_1	: 0.6192%
node_conv2d_2	: 0.0886%
node_conv2d_3	: 0.0574%
node_conv2d_4	: 0.0701%
node_conv2d_5	: 0.0486%
node_conv2d_6	: 0.0521%
node_conv2d_7	: 0.0226%
node_conv2d_8	: 0.0516%
node_conv2d_9	: 0.0366%
node_conv2d_10	: 0.0109%

图 5.7 YOLO 各层量化误差

layer_name	: SNR(N/S)
node_conv2d	: 0.0016%
node_conv2d_1	: 0.0239%
node_conv2d_2	: 0.0197%
node_conv2d_3	: 0.0123%
node_conv2d_4	: 0.0045%
node_conv2d_5	: 0.0038%
node_conv2d_6	: 0.0122%
node_conv2d_7	: 0.0027%
node_conv2d_8	: 0.0006%
node_conv2d_9	: 0.0003%
node_conv2d_10	: 0.0009%

图 5.8 YOLO 车牌检测模型逐层信噪比误差

5.3.3 LPRNet 车牌号识别模型量化误差分析

考虑到图像输入格式为 RGB888 并且使得量化模型精度损失可控，我们对输入数据的量化使用 9 比特对称量化，从而实现图像输入无需任何操作直接可用；对其它张量量化采用

10 比特对量化，以保证量化误差足够小。

LPRNet 车牌号识别模型累计信噪比误差分析如下，其中 node_conv2d_5 为输出、输出的累计误差小于 0.06%。

layer_name	: SNR(N/S)
node_conv2d	: 0.0043%
node_conv2d_1	: 0.0051%
node_conv2d_2	: 0.0073%
node_conv2d_3	: 0.0136%
node_conv2d_4	: 0.0600%
node_conv2d_5	: 0.0588%

图 5.8 LPRNet 各层量化误差

layer_name	: SNR(N/S)
node_conv2d	: 0.0110%
node_conv2d_1	: 0.0145%
node_conv2d_2	: 0.0151%
node_conv2d_3	: 0.0249%
node_conv2d_4	: 0.0130%
node_conv2d_5	: 0.0265%

图 5.9 YOLO 车牌检测模型逐层信噪比误差

参考：

https://github.com/Cat7373/STNet_LPRNet

<https://github.com/ultralytics/ultralytics>

https://gitee.com/craft/chinese_license_plate_generator

5.4 Modelsim 仿真

对系统的各个核心模块编写了测试脚本，进行仿真测试。

5.4.1 卷积计算模块

在 100 MHz 时钟下，YOLO 车牌检测模块仿真延迟约 1.7ms，对应 148.5MHz 的实际运行频率下，仿真延迟约 1.14ms。LPRNet 车牌检测模块的延迟约为 3.5ms，对应 148.5MHz 实际运行频率下延迟约 2.35ms。仿真结果体现了卷积计算采用全流水线架构极低延迟的强大优势。

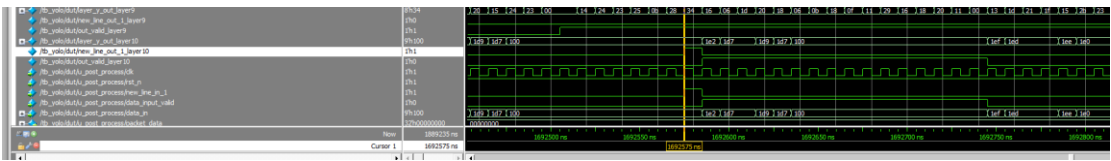


图 5.10 YOLO 模块仿真延迟（单位：ns）

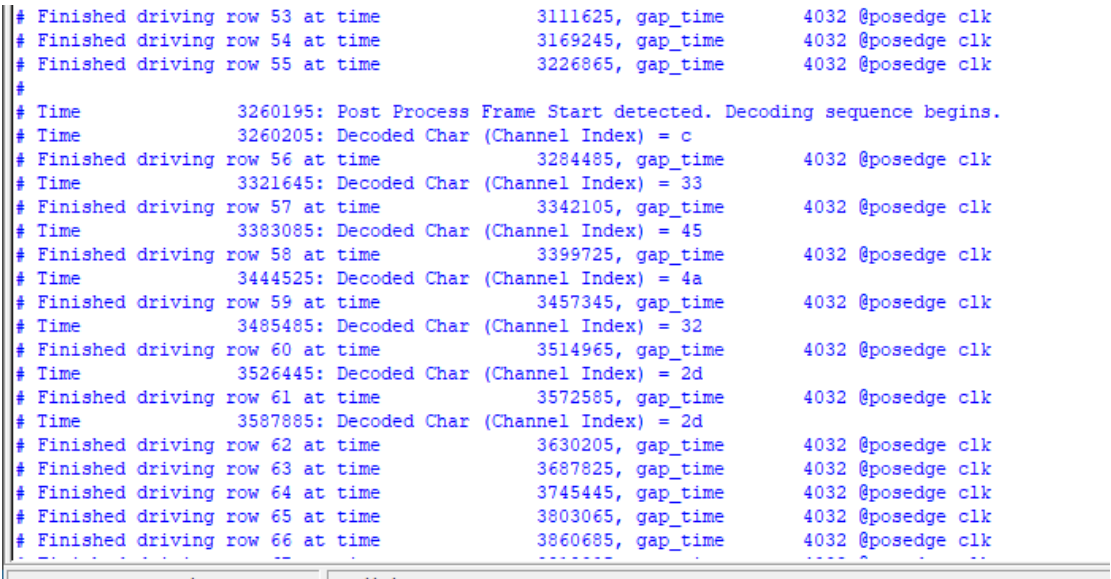


图 5.11 LPRNet 模块仿真延迟（单位：ns）

5.4.2 子图裁切缩放模块

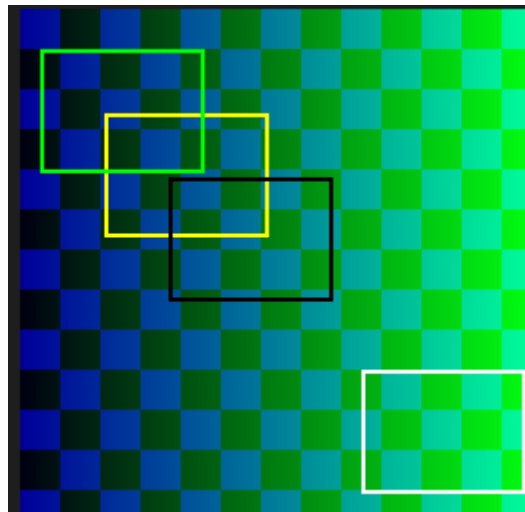


图 5.12 裁切缩放测试原图

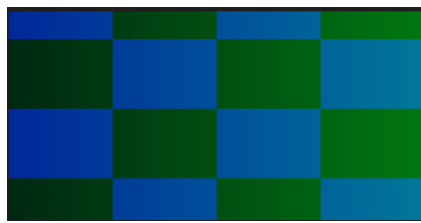


图 5.13 裁切缩放测试子图

子图的裁切、缩放功能符合预期，可以实现不同长宽的预选框到固定长度的子图的水平
和垂直裁切、缩放。

5.5 实际测试

5.5.1 延迟与帧率

输入 720P@60Hz 信号，系统能在一帧内完成多个车牌类型的识别、定位和分类，一帧内完成 8 个车牌号序列的检测。车牌类型和定位检测延迟约 16.7ms（一帧），8 个车票号检测延迟约 16.7ms（一帧）。相比于 CPU、GPU 等传统方案，系统具有低延迟的优势。

5.5.2 光强适应性



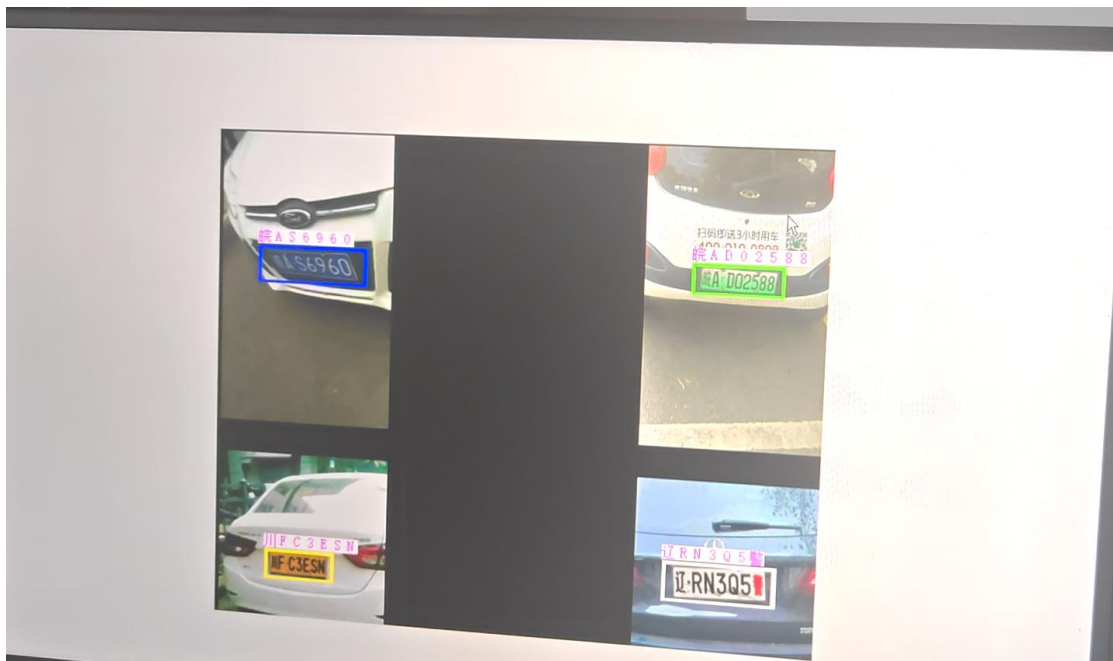
图 5.14 多种光强下的 8 个车牌号同步检测结果

上图展示了系统在不同光强下的检测适应性，在强光、弱光、逆光环境下，均能实现良好的检测效果，有着优秀的光强适应性。

5.5.3 大小、角度适应性



5.15 不同角度的较大车牌检测结果



5.16 不同角度的较小车牌检测结果

系统对车牌的大小和角度有一定的适应性，从 20*80 到 200*800 的车牌预选框大小，从 0 到 30° 左右的车牌角度（包括俯仰角、偏航角和横滚角），都可以实现良好的检测效果。

5.5.4 同步检测数量

系统在一帧内能实现 8 个车牌号的检测，也可以用轮询的方式在多帧内实现更多车牌号的检测。

5.5.5 正确率

实测车牌号的正确率，在图片清晰度高的情况下，车牌类型检测正确率十分接近 100%，车牌号检测正确率可以达到 95%以上。在清晰度不理想的情况下，车牌类型检测正确率能达到 95%以上，车牌号检测正确率能达到 80%以上。

5.5.6 资源消耗

在 PG200H 上，LUT 消耗 10.65%，Registers 消耗 27.26%，DRM 消耗 47.11%，APM 消耗 53.51%。系统的资源消耗符合预期，并有足够的余量。

GTP Usage			
Find:			
	GTP Name	Usage	
1	GTP_APM_E2	396	
2	GTP_DFF	40820	
3	GTP_DFF_C	36349	
4	GTP_DFF_CE	2344	
5	GTP_DFF_E	6885	
6	GTP_DFF_P	30	
7	GTP_DFF_PE	11	
Total LUTs: 17022 of 159800 (10.65%) LUTs as dram: 868 of 39500 (2.20%) LUTs as logic: 16154 Total Registers: 87125 of 319600 (27.26%) Total Latches: 0 DRM36K/FIFO: Total DRM = 195.5 of 415 (47.11%) APMs: Total APMs = 396.00 of 740 (53.51%) Total I/O ports = 42 of 400 (10.50%)			

图 5.17 资源消耗

5.7 时序报告

200K-mini 开发板的 HDMI 接口速率上限 60Hz@720P 下,像素时钟(pixclk_in)74.25MHz, 设定卷积处理单元时钟 (clk_pe) 频率为像素频率的两倍 145.5MHz。所有的时钟路径 Setup-Slack 和 Hold-Slack 均为正。Pixclk_in 的 Fmax 为 82.33MHz,clk_pe 的 Fmax 为 163.23MHz。系统静态时序满足所有时钟约束要求。

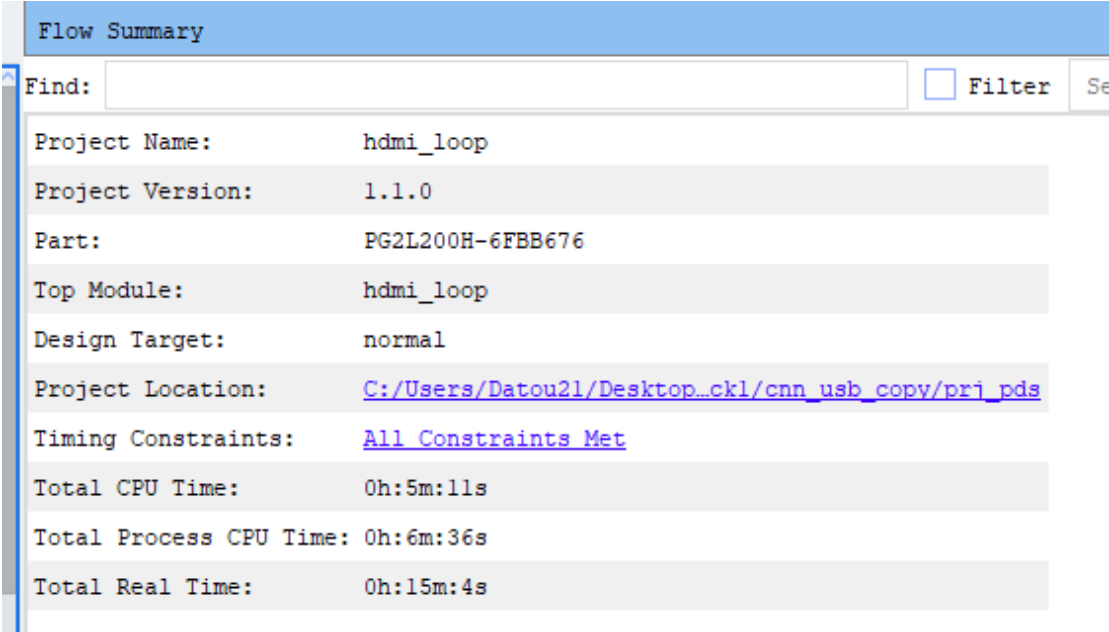


图 5.18 EDA 流程总结

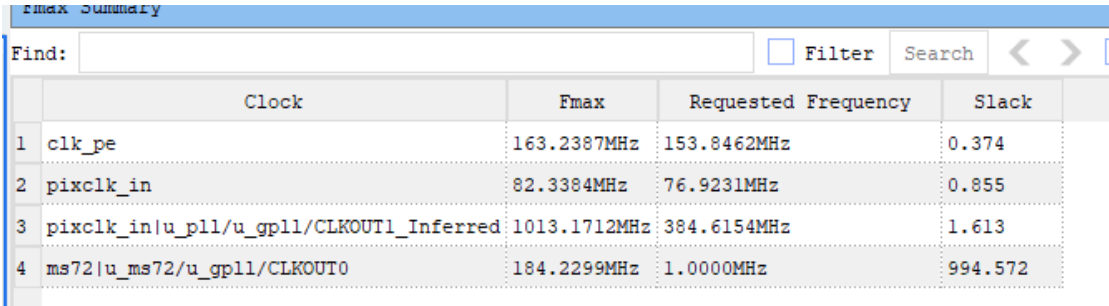


图 5.19 各时钟最大频率和 slack

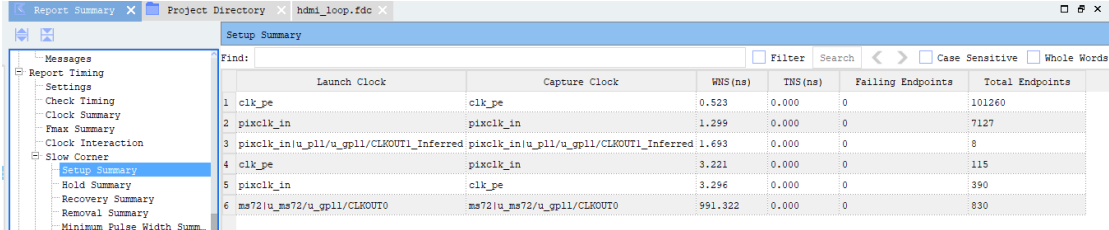


图 5.20 建立时间分析 (Slow Conner)

	Launch Clock	Capture Clock	WNS(ns)	TNS(ns)	Failing Endpoints	Total Endpoints
1	pixclk_in[u_gp11/CLKOUT1_Inferred]	pixclk_in[u_gp11/CLKOUT1_Inferred]	2.071	0.000	0	8
2	clk_pe	clk_pe	2.886	0.000	0	101260
3	clk_pe	pixclk_in	4.615	0.000	0	115
4	pixclk_in	clk_pe	4.645	0.000	0	390
5	pixclk_in	pixclk_in	6.289	0.000	0	7127
6	ms72[u_ms72/u_gp11/CLKROUT0]	ms72[u_ms72/u_gp11/CLKROUT0]	994.809	0.000	0	830

图 5.21 建立时间分析（Fast Conner）

	Launch Clock	Capture Clock	WHS(ns)	TNS(ns)	Failing Endpoints	Total Endpoints
1	clk_pe	clk_pe	0.100	0.000	0	101260
2	pixclk_in	pixclk_in	0.150	0.000	0	7127
3	ms72[u_ms72/u_gp11/CLKROUT0]	ms72[u_ms72/u_gp11/CLKROUT0]	0.161	0.000	0	830
4	clk_pe	pixclk_in	0.166	0.000	0	115
5	pixclk_in[u_gp11/CLKOUT1_Inferred]	pixclk_in[u_gp11/CLKOUT1_Inferred]	0.346	0.000	0	8
6	pixclk_in	clk_pe	0.741	0.000	0	390

图 5.21 保持时间分析（Slow Conner）

	Launch Clock	Capture Clock	WHS(ns)	TNS(ns)	Failing Endpoints	Total Endpoints
1	clk_pe	clk_pe	0.100	0.000	0	101260
2	ms72[u_ms72/u_gp11/CLKROUT0]	ms72[u_ms72/u_gp11/CLKROUT0]	0.102	0.000	0	830
3	pixclk_in	pixclk_in	0.109	0.000	0	7127
4	clk_pe	pixclk_in	0.133	0.000	0	115
5	pixclk_in[u_gp11/CLKOUT1_Inferred]	pixclk_in[u_gp11/CLKOUT1_Inferred]	0.225	0.000	0	8
6	pixclk_in	clk_pe	0.383	0.000	0	390

图 5.21 保持时间分析（Fast Conner）

5.8 功耗报告

系统的总功耗 3.377W，静态功耗 0.825W，相比于 CPU、GPU 等传统硬件，有着极大的功耗优势。

	Name	Value
1	Total On Chip Power	3.377
2	Static Power	0.825
3	External Power	0
4	Junction Temperature	48.752
5	Thermal Margin	51.248
6	Power Margin	7.279

图 5.22 系统功耗

6. AI 工具使用说明

本项目部分使用 AI 工具，优化部分 RTL 代码格式、辅助进行代码审查，辅助完成部分仿真脚本的设计。并对文档进行了一定的 AI 润色。